TRANSFLEET — SMARTLOGISTICS BACKEND

Documentation Guide

Where to look, depending on what you need to do.

Status: Weeks 1–3 of 5 complete (foundation, dispatch & delivery, events & observability) **Services:** 9, each with its own store
Tests: 340 **API:** 95 endpoints behind one gateway

Fastest way to see it running

1. Install **Docker Desktop** and **uv**, then from the repository root: `./dev.sh docker up` (Windows: `dev docker up`)
2. Load sample data: `./dev.sh docker seed`
3. Open `http://localhost:8000/docs` — one Swagger UI for every service, generated from the code. Pick a service in the top-right dropdown.

Log in with `admin@transfleet.com` / `SmartLogistics!2026`, or one of the other 10 seeded accounts (support, warehouse operators, couriers) to see what each role can and cannot do. For day-to-day coding, run the services on your machine instead: `./dev.sh infra up` then `./dev.sh start --reload`.

If you want to...

GOAL	READ	WHAT'S THERE
Set the project up and run it	`README.md`	Prerequisites, the two run modes (services on your machine or everything in Docker) on macOS, Linux and Windows, every `./dev.sh` command, configuration and troubleshooting.
Try the API myself	`README.md` → API docs `docs/curl/`	Swagger UI for all services at `localhost:8000/docs`, a Postman collection, a merged OpenAPI file and an offline HTML page with copy-paste curl commands — all regenerated from the code with `./dev.sh docs`.
Understand how it's built and why	`docs/ARCHITECTURE.md` `docs/adr/`	Services and ownership, how they communicate (REST, Kafka events, Temporal workflows), consistency and idempotency, security, observability, dispatch and delivery. Section 1.1 maps every requirement in the brief to its honest status. 23 decision records.
See the requirements and their traceability	`docs/PRD.md`	Key use cases, functional and non-functional requirements, milestones, and a matrix from each requirement to the code and tests that deliver it.
See the database design	`docs/database.md`	Every service's tables, columns, keys and constraints with ER diagrams, plus the tracking service's MongoDB collections — generated from the models, so it cannot drift.

Review what was delivered each week	`docs/phase_3.md`
	`docs/phase_2.md`
	`docs/phase_1.md`
		The build logs: what was built, how each flow behaves in each failure case, defects found and fixed, and what is deliberately left for later weeks.

Watch a dispatch run	`localhost:8233`	The Temporal UI: every dispatch and delivery workflow, step by step, with inputs, retries and failure reasons. README → *Try a dispatch* walks through one.
Check it by hand	`docs/manual-test-week3.md` `docs/manual-test-week2.md`	Step-by-step scenarios: dispatch and delivery, notifications in Mailpit, the nine metrics, one trace across services, dead letters and replay, instant sign-out.
Watch it run	`localhost:3000` · `:16686` · `:8025`	Grafana dashboards (API health, event pipeline, business KPIs), Jaeger traces, and every email the platform sent in Mailpit.
See the original requirements	`docs/projectDocs/`	The assignment briefs this project is built against, unmodified.